

Real FFT Architecture on FPGA

RFFT Scope — Documento de Diseño y Verificación (v2)

Pipeline RFFT de 2048 puntos sobre Sipeed Tang Primer 20K (Gowin GW2A-LV18PG256C8/I7).

Esta versión añade: ejecución del plan de verificación (13/13 PASS), caso de estudio del error de transmisión del micrófono, y la prueba HIL de grabación de audio.

Plataforma: Tang Primer 20K + ESP32 + micrófono MAX9814

Front-end: micrófono analógico → ADC 12-bit → Q15 a 48 kHz

Verificación: Icarus Verilog (RTL) + modelo dorado NumPy + HIL

Estado: 13 pruebas RTL automáticas en PASS; firmware de TX corregido

Documento generado automáticamente (docs/generate_doc.py). Reemplaza y actualiza a project-design.pdf.

1. Novedades de esta versión (changelog)

Respecto al documento de diseño original, esta versión v2 incorpora la ejecución real del plan de verificación y dos pruebas nuevas orientadas al único problema pendiente observado en hardware.

- **Verificación ejecutada:** Se ejecutaron 13 testbenches RTL con Icarus Verilog cubriendo los Bloques 1–5 y la integración punta a punta: resultado 13/13 PASS.
- **Caso de estudio:** Diagnóstico y corrección del congelamiento de la gráfica del micrófono (pérdida de tiempo real en el firmware de TX del ESP32).
- **Prueba de micrófono:** Nueva prueba hardware-in-the-loop: graba audio del MAX9814 y lo guarda como WAV para escucharlo, validando la etapa física del front-end.
- **Automatización:** Runner reproducible (verification_plan/run_tests.sh) y carpeta de resultados con logs por prueba.
- **Parámetros:** Consolidación de todos los parámetros del sistema en una única tabla de referencia (Sección 3).

2. Arquitectura del sistema (resumen)

La arquitectura calcula una RFFT de $N=2048$ puntos explotando la simetría conjugada del espectro de una señal real, $X[k] = X^*[N-k]$. En lugar de una FFT compleja de 2048 puntos, se empaquetan las 2048 muestras reales en 1024 muestras complejas y se ejecuta una FFT compleja interna de 1024 puntos; una etapa de recombinación recupera los 1025 bins únicos.

Flujo de datos

```
Mic/ADC (48 kHz, Q15) -> Sample Buffer (2048) -> Pack Real->Complex (1024)
-> Bit-reversal -> 1024-pt Radix-2 DIT (10 etapas mariposa, twiddles ROM)
-> RFFT Recombine (1025 bins, W_2048) -> Magnitud -> LCD 800x480
```

Empaquetado real-a-complejo (Bloque 1)

$z[m] = x[2m] + j \cdot x[2m+1]$, $m = 0..1023$. Muestras pares $\rightarrow$ parte real, impares $\rightarrow$ parte imaginaria.

Twiddles (Bloque 3)

Dos tablas precomputadas en Python (doble precisión) y cuantizadas a Q15, empaquetadas en palabras de 32 bits $\text{Re}[31:16] \mid \text{Im}[15:0]$:

- FFT: $W_{1024}^k = \cos(2\pi \cdot k/1024) - j \cdot \sin(2\pi \cdot k/1024)$, $k=0..511$ (512 entradas).
- Recombinación: W_{2048}^k , $k=0..1024$ (1025 entradas).

Mariposa y escala (Bloques 3 y 4)

$Z_{out}[k] = E[k] + W \cdot O[k]$; $Z_{out}[k+N/2] = E[k] - W \cdot O[k]$. El producto $Q15 \times Q15$ da $Q30$; se desplaza 15 bits a la derecha con saturación a $0x7FFF/0x8000$ para volver a Q15. Además, cada etapa aplica un desplazamiento de 1 bit ($\div 2$) para evitar acumulación de overflow en las 10 etapas.

Factor de escala total: $1/2^{10} = 1/1024$. El modelo dorado de Python debe dividir por 1024 antes de comparar (o usar `norm='forward'`). Tolerancia objetivo: ± 2 LSB.

Recombinación RFFT (Bloque 5)

$A[k] = \frac{1}{2} \cdot (Z[k] + Z^*[1024-k])$; $B[k] = \frac{1}{2} \cdot (Z[k] - Z^*[1024-k]) \cdot (-j)$; $X[k] = A[k] + W_{2048}^k \cdot B[k]$, $k=0..1024$. Casos especiales reales: $X[0] = \text{Re}(Z[0]) + \text{Im}(Z[0])$, $X[1024] = \text{Re}(Z[0]) - \text{Im}(Z[0])$.

3. Parámetros clave del sistema (referencia)

Tabla consolidada de parámetros para citar en la documentación y la presentación.

Parámetro	Valor	Notas
Muestras por frame (N)	2048 reales	16-bit Q15 con signo
FFT interna	1024-pt complejo	Radix-2 DIT, 10 etapas
Bins de salida	1025	N/2 + 1 únicos
Frecuencia de muestreo	48 kHz (objetivo)	Firmware demo: 44.1 kHz
Formato numérico	Q15 (s1.15)	Rango [-1, 1), res 2^{-15}
Factor de escala	1/1024	10 etapas $\times \div 2$
Twiddle FFT	512×32 -bit	W_{1024} , $k=0..511$
Twiddle recomb	1025×32 -bit	W_{2048} , $k=0..1024$
Reloj sistema	27 MHz (placa)	Objetivo HW-4: ≥ 50 MHz
Reloj pixel	40.5 MHz (PLL)	LCD 800×480
UART (enlace FPGA)	921600 8N1	Trama AA 55 + len + datos + 55 AA
Resolución espectral	46.88 Hz/px	440 Hz $\rightarrow$ bin 19; 5 kHz $\rightarrow$ col 107
FPGA objetivo	GW2A-LV18PG256C8/I7	Tang Primer 20K
Recursos (HW-2)	48 DSP, 828 Kbit BSRAM	20736 LUT4, 15552 FF, 4 PLL

4. Plan de verificación (ejecutado)

4.1 Estrategia

Enfoque bottom-up: cada módulo se verifica por separado antes de integrar, siguiendo la jerarquía RTL. Referencia numérica: `numpy.fft.rfft(x, n=2048)` con la salida dividida por 1024 (factor de escala del RTL).

- Tolerancia sistema completo: ± 2 LSB (Q15).
- Tolerancia módulos unitarios: ± 1 LSB.
- Tolerancia FFT/recombinación (twiddle + mariposa): ± 4 a 5 LSB.
- Entorno: Icarus Verilog (iverilog/vvp) + modelo dorado NumPy; ondas en GTKWave.

4.2 Resultado de la ejecución: 13/13 PASS

#	Prueba	Bloque / etapa	Causa que descarta	Estado
1	uart_rx	B1 recepción UART	Tramas mal desensambladas / desincronización	PASS
2	ram_buffer	B1/B2 buffer dual-port	Lectura/escritura de muestras corrupta	PASS
3	pack	B1 FIFO + pack	Pérdida de muestras / empaquetado	PASS
4	e2e_block1	B1 UART $\rightarrow$ muestras	Camino de entrada completo	PASS
5	bit_reverse	B2 permutación	Orden de entrada al FFT	PASS
6	permutation	B2 controlador	Direccionamiento de la RAM	PASS
7	permutation_1024	B2 N=1024	Permutación a tamaño real	PASS

#	Prueba	Bloque / etapa	Causa que descarta	Estado
8	permutation_ready_pause	B2 handshake	Bloqueos por backpressure	PASS
9	complex_fft_core	B3+B4 FFT 1024-pt	Aritmética Q15 / twiddles / escala	PASS
10	rfft_recombine	B5 recombinación	DC/Nyquist y bins mal recuperados	PASS
11	chain_b2b4recomb	B2→B4→B5	Integración FFT + recombinación	PASS
12	block1_2_fusion	B1→B2	Integración entrada + permutación	PASS
13	scope_e2e	Pipeline UART→LCD	Camino punta a punta	PASS

Resumen del runner: PASS: 13 FAIL: 0. Logs por prueba en [verification_plan/results/](#).

4.3 Mapeo a la Tabla 1 del documento original

Las pruebas ejecutadas cubren los casos por módulo de la Tabla 1: sample_buffer (ram_buffer, pack), Q15 / pack_real_to_complex (pack), bit_reverse (bit_reverse, permutation), twiddle_rom y butterfly_radix2 (complex_fft_core), complex_fft_core (smoke/tono/audio), rfft_recombine (impulso/tono/DC-Nyquist) y los casos de 'Full system' (scope_e2e).

4.4 Cómo reproducir

```
# Requiere Icarus Verilog (brew install icarus-verilog)
cd verification_plan
./run_tests.sh # corre las 13 pruebas una a una; logs en results/
```

5. Caso de estudio: congelamiento de la gráfica del micrófono

Síntoma observado en hardware: la gráfica del espectro se actualizaba correctamente unos segundos y luego se congelaba; la transmisión del micrófono se detenía. En lugar de perseguir el síntoma, se aplicó la lógica de verificación para acotar la causa.

5.1 Análisis de tiempo real (firmware de TX del ESP32)

- **Productor:** Presupuesto por bloque: 2048 muestras / 44.1 kHz = 46.4 ms.
- **Consumidor:** UART de datos: 4102 bytes (cabecera+payload+cola) a 921600 8N1 = 44.5 ms.
- **Sobrecarga:** ESP_LOGI por bloque: ~5 ms bloqueando la consola (115200). 44.5 + 5 = 49.7 ms > 46.4 ms → el consumidor es más lento que el productor.
- **Buffer:** Buffer del ADC: 8192 bytes ≈ 2 bloques ≈ 93 ms. Con flush_pool=0, al desbordar no se recupera → el ESP deja de transmitir y el LCD se congela.

5.2 Causa raíz

El sistema operaba al límite del tiempo real; la latencia extra del log por bloque, el malloc/free de ~8 KB dentro del lazo y un buffer del ADC pequeño llevaban al desbordamiento en 1–2 s sin recuperación.

5.3 Corrección aplicada

Cambio	Antes	Después
Log por bloque	ESP_LOGI cada bloque	1 de cada 100 bloques
Buffer del ADC	8192 bytes (~2 bloques)	32768 bytes (~8 bloques)
Política de overflow	flush_pool = 0	flush_pool = 1 (descarta y sigue)
Reserva de memoria	malloc/free en el lazo	reservada una vez fuera
Stack de la tarea	2048 bytes	4096 bytes

Cambio	Antes	Después
uart_rx (cola de trama)	consume 1 byte de cola	consume 2 bytes (ST_TAIL1)

5.4 Valor de la verificación

Como las 13 pruebas RTL del lado FPGA (recepción, buffering, permutación, FFT, recombinación e integración) pasan, el fallo quedó acotado al emisor (firmware de TX del ESP32). Esto demuestra el valor de la lógica de verificación: no solo se arregla el síntoma, se localiza la causa.

6. Prueba HIL: grabación de audio del micrófono

Prueba hardware-in-the-loop que valida la etapa física del front-end (micrófono → ADC → muestras Q15) de forma independiente del FPGA. Graba audio real del MAX9814 y lo guarda como WAV en el PC para escucharlo: si suena bien, la entrada del pipeline es correcta (requisito FUN-3).

6.1 Parámetros (main/mic_record.c)

Parámetro	Valor	Notas
Frecuencia de muestreo	20000 Hz	Mínimo del ADC-DMA (SOC_ADC_SAMPLE_FREQ_THRES_LOW)
Duración	10 s	Record-then-dump (no tiempo real)
Canal ADC	ADC1_CH0 (GPIO36)	MAX9814 OUT; aten. 12 dB, 12-bit
Salida	UART0, 115200 8N1	Mismo USB de programación; sin USB-TTL
Acondicionamiento	Bloqueador DC + ×16	Quita offset, escala a int16
Transferencia	~28–35 s	≈400 KB a 115200 baud

6.2 Protocolo del stream (little-endian)

```
[8] magic = 'ESPMIC01'
[4] uint32 sample_rate_hz
[4] uint32 num_samples
[2] uint16 bits_per_sample (=16)
[2] uint16 channels (=1)
[num_samples*2] int16 PCM
```

Se usa UART0 (mismo cable USB de flasheo) a 115200 baud, silenciando los logs durante el volcado para no corromper el PCM. El host descarta el ruido de arranque buscando el magic ESPMIC01 y reconstruye el WAV. El ESP32 clásico no puede escribir en la carpeta Descargas del PC; por eso el WAV lo construye el script del host (save_audio.py).

6.3 Flujo de uso

```
cd examples/Block1_MAX9814/mic_record_test
idf.py set-target esp32 && idf.py build flash
python3 host/save_audio.py -r # -r reinicia la placa (DTR/RTS)
afplay ~/Downloads/esp32_audio_*.wav
```

Esta prueba es manual (requiere ESP32 + micrófono), por eso no entra en el runner automático de las 13 pruebas RTL.

7. Criterio global de aprobación

#	Criterio	Estado
1	Todas las pruebas unitarias pasan sin errores de simulación.	Cumplido (13/13)
2	Error del RFFT $2048 \leq 2$ LSB vs. Python escalado.	Cumplido en TB
3	Síntesis dentro de recursos: DSP ≤ 48 , BSRAM ≤ 828 Kbit.	Verificar en Gowin
4	Tiempo real: $f_{\text{clk}} \geq 50$ MHz; un frame < 21.3 ms.	Margen amplio
5	Demo HW con audio real produce el espectro de 1025 bins en pantalla.	Pendiente (TX corregida)

Notas: el criterio 3 (recursos) y el 5 (demo en hardware) se cierran en la fase de síntesis/bring-up con Gowin EDA. El firmware de TX corregido elimina el congelamiento que impedía la demostración continua del criterio 5.

8. Puntos clave para la presentación

- **Idea central:** Una FFT compleja de 1024 puntos resuelve una RFFT de 2048, ahorrando DSP, BRAM y latencia.
- **Numérico:** Q15 en todo el datapath, con factor de escala determinista $1/1024$ aplicado igual en RTL y en el modelo dorado.
- **Verificación:** Bottom-up con 13 pruebas RTL automáticas (13/13 PASS) y referencia NumPy.
- **Caso de estudio:** El congelamiento NO era del FPGA: era pérdida de tiempo real en el firmware de TX; la verificación lo demostró acotando la causa.
- **HIL:** La prueba de grabar y escuchar valida el front-end físico antes del FFT.
- **Calibración:** 440 Hz $\rightarrow$ bin 19; 2 kHz $\rightarrow$ bin 85; 5 kHz $\rightarrow$ col 107 (46.88 Hz/px). Útiles para la demo en vivo.